

DSTS — Implementation Plan

Team: 4 (Persons A–D) · **Code freeze:** 29 Aug · **Review:** 30 Aug **Goal:** working end-to-end system with a queryable database

1. Reference papers — read your lane's before you code

Paper	Governs	What to read
Mohan & Menon (2020) , <i>IET Computer Vision</i> 14(7):426–433	DSTS/BSTS model, state transition equations	The transition function; probabilities sum to 1 across zones
Menon, Jayaraman & Govindaraju (2011) , <i>Computer</i> 44(9):73–79 — <i>Three R's of Cyberphysical Spaces</i>	Overall architecture: Recognition → Reasoning → Retrieval	The three-stage framing; precision/recall methodology
Menon, Jayaraman & Govindaraju (2012) , <i>Procedia CS</i> 10:366–373 — <i>Spatio-Temporal Querying in Smart Spaces</i>	The database schema and the queries	§3 <i>Definition (State Relation)</i> and <i>(Occupancy Relation)</i> ; Queries 1–2 give the SQL
Menon, Jayaraman & Govindaraju (2014) , <i>JAIHC</i> 5(3):383–392 — <i>Probabilistic Spatio-Temporal Retrieval</i>	Extended version	Point vs interval representation; §5 query-dependent precision/recall
Chauhan & Batra (2014) , <i>ICACCCT</i> pp.1662–1666 — <i>Finding Similar Items Using LSH and Bloom Filter</i>	The routing layer	LSH + Bloom for set-membership

Required reading: A → Chauhan & Batra + Mohan & Menon · B → Three R's §5 + Menon 2014 §5 · C → Menon 2012 §3 and Menon 2014 §3 · D → Three R's (Retrieval)

2. Lanes

Lane	Person	Owns	Load
Biometric Pipeline & Routing — the contribution	A	contracts.py , testing.py , data/ , routing/	5 days
Events & Evidence	B	events/ , eval/ , tests/test_integration.py	4 days
State & Queries — critical path	C	state/	4 days

Lane	Person	Owns	Load
Wiring & Interface	D	orchestration/ , security/ , api/ , scripts/	4.5 days

Why A owns the entire biometric chain: face image → embedding → LSH code → Bloom bitmap → ranked building is one continuous pipeline. A weakness anywhere in it kills the contribution, and the failure is only visible end to end. One person owns whether it works, with no handoff in the middle.

Fallbacks — none of them are A:

```
A slips → whole team stops. Nothing else matters if the contribution fails.
B slips → D absorbs (D drops api/)
C slips → B moves across the same day
D slips → orchestration/ goes to B; api/ and access.py dropped
```

3. Every file, and what it is for

Shared — Person A, written 23 Aug

`dsts/contracts.py` · ~90 lines · FROZEN after 23 Aug

The single definition of every value that crosses a lane boundary. If you want to pass a raw `dict` between lanes, add a dataclass here instead.

```
EMBED_DIM = 512
N_ZONES = 9
TRANSITION_ZONE = "zT"

@dataclass(frozen=True)
class Event:
    seq: int
    time: datetime
    building_id: str
    zone: str
    embedding: np.ndarray          # (512,) float32, L2-normalised
    ground_truth: str | None      # simulation only

@dataclass(frozen=True)
class Candidate:
    occupant_id: str
    probability: float

@dataclass(frozen=True)
class Recognition:
    candidates: list[Candidate]   # sums to <= 1
    accepted: bool
    best_id: str | None
```

```

@dataclass(frozen=True)
class BuildingRank:
    building_id: str
    hits: int # out of L tables

@dataclass(frozen=True)
class QueryRequest:
    kind: str # "singleton" | "duration" | "snapshot"
    role: str # "officer" | "analyst" | "self"
    occupant_id: str | None = None
    zone: str | None = None
    t_start: datetime | None = None
    t_end: datetime | None = None

@dataclass(frozen=True)
class QueryResponse:
    rows: list[dict]
    redacted: bool

class Transport(Protocol):
    def send(self, building_id: str, op: str, payload: dict) -> dict: ...

```

`dsts/testing.py` · ~80 lines

Throwaway stand-ins so four lanes run in parallel instead of queuing.

- **FakeEmbedder** — returns made-up 512-vectors, but *consistently*: same identity → nearly the same vector, different identities → far apart. Lets routing and orchestration be built with no torch installed.
- **InMemoryStore** — same method names as `SqliteStore`, backed by a list. Lets D build before C's database exists.
- **FakeRecogniser** — returns canned `Recognition` objects from a script.

The rule: one test file must pass against both the fake and the real thing. That test is what turns integration day into a one-line swap.

Lane A — `data/` and `routing/`

`dsts/data/dataset.py` · ~100 lines

Turns a folder of face images into a campus assignment.

- Walks the dataset directory, finds identities with enough photos
- Splits each identity's photos into **20 reference + 20 test, disjoint**
- Assigns identities to buildings (10 × 50)
- Reserves two extra pools: a **µ pool** (never enrolled, used to compute the centring vector) and a **stranger pool** (never enrolled, for open-set testing)

```
def discover_identities(root: Path, min_images: int) -> list[Identity]
def build_campus(identities, n_buildings, per_building) -> Campus
```

Why the μ pool matters: face embeddings cluster in a narrow cone, so you subtract a population mean before measuring angles. If that mean came from enrolled occupants you'd be quietly pooling the exact data the design claims never to centralise.

`dsts/data/embeddings.py` · ~90 lines

Converts face images to normalised 512-D vectors.

```
class FaceEmbedder:
    def embed_paths(self, paths: list[str], batch_size: int = 64) -> np.ndarray
```

Resizes to the model's input size, standardises pixels, runs the network in batches, **L2-normalises** the output. Normalisation is what makes similarity purely a question of angle — everything downstream depends on it.

`dsts/data/enroll.py` · ~110 lines

CLI that runs `dataset` + `embeddings` **once** and saves everything, so nobody recomputes 20,000 embeddings.

```
python -m dsts.data.enroll --dataset-root ... --out artifacts/enrollment.npz
```

Writes `enrollment.npz` containing: reference embeddings + owner index, test embeddings + owner index, the μ vector, the building assignment, the stranger pool.

`dsts/data/calibrate.py` · ~110 lines · the gate

Measures whether the dataset actually works with LSH, and derives the parameters.

```
def measure_angles(ref, owner_idx) -> (genuine_angles, impostor_angles)
def derive_parameters(p1, p2, N, delta) -> (k, L)
```

Computes the mean angle between photos of the **same** person and between **different** people. From those: $p1 = 1 - \theta_{\text{gen}}/180$, $p2 = 1 - \theta_{\text{imp}}/180$, then $k^* = \log(N)/\log(1/p2)$ and $L^* = \log(\delta)/\log(1 - p1^k)$.

If genuine and impostor angles don't separate, LSH routing cannot work. This file is how you find that out on day 2 rather than day 6.

`dsts/data/recognition.py` · ~70 lines

Compares a probe against a building's gallery.

```
class Recogniser:
    def recognise(self, probe: np.ndarray, gallery: Gallery) -> Recognition
```

Takes the **minimum distance across all of an occupant's templates** (never an average — an averaged centroid drifts toward whichever appearance dominates). Converts distances to probabilities via softmax:

$$p(o_i) = \exp(-\lambda \cdot d_i) / \sum \exp(-\lambda \cdot d_j) .$$

Returns a **distribution, not a hard decision** — that's the Three R's Recognition stage, and it's what the state layer consumes.

`dsts/routing/simhash.py` · ~70 lines

Generates the shared hyperplane bank and turns embeddings into codes.

```
class HyperplaneBank:
    @classmethod
    def generate(cls, L: int, k: int, dim: int, seed: int) -> HyperplaneBank
    def signatures(self, X: np.ndarray) -> np.ndarray    # (n, L) integer codes
```

Dots each embedding against `k` random hyperplanes per table, keeps the **sign** of each as one bit, packs the `k` bits into an integer. Repeats for `L` tables.

`dsts/routing/bloom.py` · ~100 lines

Sizes, builds, and queries a Bloom filter.

```
def bitmap_params(n_items: int, fp_rate: float) -> tuple[int, int]    # (m, h)

class BloomBitmap:
    def insert(self, item: bytes) -> None
    def might_contain(self, item: bytes) -> bool
```

Sizing: `m = -n · ln(p) / (ln2)2` bits, `h = (m/n) · ln2` positions per item. Each item's positions come from a cryptographic hash called `h` times with different salts. **Never a false negative** — bits only go 0→1.

`dsts/routing/summary.py` · ~80 lines

Turns one building's gallery into the bitmap it ships to every peer.

```
def build_summary(codes: np.ndarray, fp_rate: float) -> BloomBitmap
def save(bitmap, path) / def load(path) -> BloomBitmap
```

Deduplicates `(table_index, code)` pairs **per table** — the same code number in two different tables is two different items — then inserts each into the bitmap.

`dsts/routing/ranker.py` · ~70 lines

The routing decision. **Zero network messages**.

```
class Ranker:
    def rank(self, probe: np.ndarray) -> list[BuildingRank]
```

Hashes the probe into its L codes, tests each against every peer's **cached** bitmap, tallies hits per building, returns them ranked.

Lane B — `events/` and `eval/`

`dsts/events/mobility.py` · ~150 lines

Generates a realistic synthetic event stream.

```
class MobilityModel:
    def generate(self, n_events: int, seed: int) -> list[Event]
```

Occupants move between **adjacent** zones only (never teleporting), using the zone graph from `state/zones.py`. Occasionally an occupant exits via `zT` and travels to another building.

`dsts/events/scenario.py` · ~80 lines

Builds the hand-crafted demo walk used on review day.

```
def build_demo_walk() -> list[Event]
```

~40 events, fully deterministic: one occupant registered at B1 moves through zones, exits via `zT`, appears at B5, gets routed and admitted as a visitor.

`dsts/eval/metrics.py` · ~130 lines

Computes the numbers.

```
def paper_metrics(events, theta_grid) -> dict      # precision/recall vs  $\theta$ 
def routing_metrics(lookups) -> dict              # rank-1 %, buildings contacted
```

`dsts/eval/report.py` · ~80 lines

Renders the charts: precision/recall vs θ , and buildings contacted vs broadcast.

`tests/test_integration.py` · ~100 lines

The end-to-end test. Runs the demo scenario through the full stack.

Lane C — `state/`

`dsts/state/zones.py` · ~90 lines · delivered 23 Aug

The floorplan. 8 interior zones plus the transition zone `zT`.

```

ZONES = ["z1", ..., "z8", "zT"]
ZONE_ADJACENCY = {"zT": ["z1"], "z1": ["zT", "z2", "z3"], ...}

def neighbours(zone) -> list[str]
def zone_hops(a, b) -> int
def validate() -> None

```

`dsts/state/schema.sql` · ~40 lines

The four relation tables. **Defined in Menon 2012 §3 — implement what the paper specifies, don't invent it.**

```

CREATE TABLE registered_state (
    time TEXT, occupant TEXT, zone TEXT, probability REAL,
    PRIMARY KEY (time, occupant, zone)
);
CREATE TABLE registered_occupancy (
    start_time TEXT, occupant TEXT, zone TEXT, end_time TEXT, probability REAL,
    PRIMARY KEY (start_time, occupant, zone)
);
CREATE TABLE visitor_state      ( /* same shape */ );
CREATE TABLE visitor_occupancy ( /* same shape */ );

```

`dsts/state/store.py` · ~150 lines

SQLite persistence, one `.db` file per building.

```

class SqliteStore:
    def write_state(self, time, occupant, zone, probability) -> None
    def read_state(self, time, occupant) -> list[Row]
    def read_occupancy(self, occupant, t_start, t_end) -> list[Row]

```

Occupancy is derived on write, in the same transaction as state.

`dsts/state/bsts.py` · ~150 lines

The state transition. Governed by Mohan & Menon.

```

class StateTable:
    def apply(self, detected_zone: str, event_probs: dict[str, float]) -> None
    def verify_constraint(self) -> None

```

For the zone where the event occurred: $z_j = p + x \cdot z_j$. For every other zone: $z_l = x \cdot z_l$, where $x = 1 - p$.

`dsts/state/queries.py` · ~120 lines

The three query types. SQL forms are in Menon 2012, Queries 1–2.

```
def singleton(store, occupant, zone, t) -> float
def duration(store, occupant, t_start, t_end) -> list[Interval]
def snapshot(store, zone, t, theta) -> list[Candidate]
```

Lane D — `orchestration/`, `security/`, `api/`, `scripts/`

`dsts/orchestration/building.py` · ~120 lines

One building node. Wires everything together for a single building.

```
class Building:
    def __init__(self, building_id, gallery, store, ranker, transport)

    def handle_event(self, event: Event) -> None:
        """My camera saw a face. Identify, then update state."""

    def identify_probe(self, embedding) -> Recognition:
        """Another building is asking: is this one of yours?"""
```

`dsts/orchestration/campus.py` · ~70 lines

Boots N buildings and plays an event stream through them.

```
class Campus:
    def run(self, events: list[Event]) -> None
```

`dsts/security/transport.py` · ~60 lines

How one building reaches another.

```
class InProcessTransport:
    def send(self, building_id: str, op: str, payload: dict) -> dict
```

`dsts/security/access.py` · ~80 lines

Role-based filtering of query results.

```
def apply_policy(response: QueryResponse, role: str) -> QueryResponse
```

`dsts/api/server.py` · ~120 lines

FastAPI. Serves the dashboard, exposes a query endpoint, streams occupancy updates as events process.

`dsts/api/static/index.html` · ~150 lines

The dashboard: building list, live occupancy table, query box, role selector.

`scripts/run_demo.sh` · ~20 lines

One command that replays the scripted walk to the terminal.

4. Repository structure

```
dsts/
├── README.md · OWNERSHIP.md · pyproject.toml
├── config/campus.yaml
├── dsts/
│   ├── contracts.py                A
│   ├── testing.py                 A
│   ├── data/                      dataset · embeddings · enroll · calibrate · recognition  A
│   ├── routing/                   simhash · bloom · summary · ranker                A
│   ├── events/                    mobility · scenario                            B
│   ├── eval/                      metrics · report                             B
│   ├── state/                     zones · schema.sql · store · bsts · queries        C
│   ├── orchestration/             building · campus                             D
│   ├── security/                  transport · access                             D
│   └── api/                       server · static/index.html                    D
├── tests/ (mirrors the package; test_integration.py owned by B)
├── scripts/run_demo.sh             D
└── artifacts/ # gitignored
```

5. Conventions

Naming: modules `snake_case` singular · functions `snake_case` verb-first · classes `PascalCase` · constants `UPPER_SNAKE` · private `_leading` · tests `test_<behaviour>` · booleans `is_ / has_`

Vocabulary: `occupant` · `building` · `zone` · `probe` · `gallery` · `visitor` · `summary` ·

Limits: line 100 · function 40 lines · file 300 lines · 4 parameters · nesting depth 3

Style: type hints on every public function · module docstring on every file · `pathlib` not `os.path` · f-strings only · never bare `except:` · no mutable defaults · constants at module top

```
[tool.ruff]
line-length = 100
target-version = "py310"
[tool.ruff.lint]
select = ["E", "F", "W", "I", "N", "UP", "B", "SIM", "C90"]
```

```
[tool.ruff.lint.mccabe]
max-complexity = 10
```

Before every commit: `ruff check . --fix && ruff format . && pytest -q`

6. Day by day

22–23 Aug · setup — everyone

- ☐ Repo, pyproject.toml, ruff, pytest green on an empty test
- ☐ contracts.py written and FROZEN (A)
- ☐ testing.py – the three fakes (A)
- ☐ state/zones.py delivered (C)
- ☐ config/campus.yaml (all agree)
- ☐ Vocabulary pinned in README (all agree)
- ☐ OWNERSHIP.md committed and acknowledged (all)
- ☐ Dataset shared, or B/C/D confirm they don't need it
- ☐ Everyone has read their lane's papers

Person A — Biometric Pipeline & Routing

Day	Task	Done when
24	dataset.py, embeddings.py, 10-identity smoke test	Real vectors produced; genuine angle visibly below impostor
25	enroll.py → enrollment.npz, then calibrate.py → k, L ⚠	Histograms separate; parameters derived. This is the gate
26	simhash.py, bloom.py	Same seed → identical planes; no false negatives on 10k inserts
27	summary.py, ranker.py, whole-building test	Correct building ranked first ≥85% at full scale
28	recognition.py + integration	—
29	Fix, freeze, rehearse	—

Person B — Events & Evidence

Day	Task	Done when
24	mobility.py — zone walk over the adjacency graph	Occupants only move between adjacent zones
25	mobility.py — inter-building travel + seed reproducibility	Same seed twice → identical event list
26	scenario.py → the demo walk	~40 events, B1 → B5, hand-checkable

Day	Task	Done when
27	<code>eval/metrics.py</code> + <code>eval/report.py</code>	Precision/recall curve; buildings-contacted number
28	<code>tests/test_integration.py</code> + integration	End-to-end test passes
29	Fix, freeze	—

Person C — State & Queries · critical path

Day	Task	Done when
24	<code>schema.sql</code> + <code>store.py</code> (skeleton)	Four tables created; can write and read a row
25	<code>store.py</code> complete + swappability test ⚠️	Same test passes on <code>InMemoryStore</code> and <code>SqliteStore</code>
26	<code>bsts.py</code> + occupancy derivation	Rows sum to 1; state and occupancy written together
27	<code>queries.py</code> — the three query types	Correct rows against the demo scenario
28	Integration	Real store replaces the fake
29	Fix, freeze	—

Person D — Wiring & Interface

Day	Task	Done when	Droppable?
24	<code>transport.py</code> + <code>building.py</code>	A Building can call a peer	No
25	<code>campus.py</code> + <code>run_demo.sh</code>	10 buildings boot; scenario replays to terminal	No
26	<code>api/server.py</code> + <code>index.html</code>	Page loads, shows buildings and occupancy	Yes
27	<code>access.py</code> — three roles	Same query, three different answers	Yes
28	Integration	UI reads from the real store	—
29	Fix, freeze	—	—

28 Aug · integration — everyone

Swap fakes for real components. Run the scenario end to end. Fix. No new features.

29 Aug · freeze

Full run, capture numbers, **rehearse twice, timed**. Code freezes at midnight.

30 Aug · review

7. Final output — what must exist on 29 Aug

```
git clone <repo> && cd dsts
pip install -e ".[dev,ml]"
python -m dsts.data.enroll      # builds enrollment.npz
bash scripts/run_demo.sh       # terminal walk-through
python -m dsts.api.server       # browser opens
```

Four things must work:

1. The scripted walk — one occupant, Building 1 → Building 5:

```
B1 event z1 → recognised locally: 0217
B1 event zT → transition probability rising
B1 evicted → 0217 has left
B5 event z1 → no local match
B5 bloom rank → B1 (187/256), B3 (3/256), others 0
B5 → B1 → confirmed: 0217
B5 → visitor record created
```

2. Three queries returning correct rows:

```
"Was 0217 in zone z2 at 09:31?" → 0.61
"Where was 0217 between 09:00-10:00?" → interval track across both buildings
"Who was in B5 zone z1 at 09:47?" → 0217 (visitor), 0403, 0411
```

3. Real rows in all four tables. Open the `.db` and show them.

4. The numbers: recognition accuracy · home building ranked first % · buildings contacted per stranger vs broadcast.

8. OWNERSHIP.md — commit 23 Aug

```
| Files | Person |
|---|---|
| contracts.py, testing.py, data/, routing/ | A |
| events/, eval/, tests/test_integration.py | B |
| state/ | C |
| orchestration/, security/, api/, scripts/ | D |

## Fallbacks
- A slips → whole team stops; nothing else matters if the contribution fails
- B slips → D absorbs (D drops api/)
- C slips → B moves across the same day
- D slips → orchestration to B; api/ and access.py dropped
```

Acknowledged by all four on 23 Aug.

9. Drop order if behind

1. `eval/report.py` charts → raw numbers from the run summary
2. `access.py` role filtering → demo one role, describe the rest
3. Web UI → terminal output from `run_demo.sh`
4. Snapshot query → the other two carry the point
5. `mobility.py` statistical run → scripted scenario only
6. 10 buildings → 5 buildings

Never drop: the four tables with real rows · the buildings-contacted number · the scripted B1→B5 walk.